

Introdução às Redes de Comunicação

Ficha de exercícios

WINSOCK UDP

Em termos globais, um sistema distribuído é constituído por duas ou mais aplicações que correm em dispositivos distintos interligados por uma rede de dados e que trocam informação (*bytes*) entre si através desta. Qualquer dispositivo ligado a uma rede possui, além de *hardware* específico (interface/placa de rede), *software* que implementa um conjunto de protocolos/regras de comunicação (subsistema de comunicação e aplicações). Na Internet, este conjunto de protocolos constitui a denominada pilha protocolar TCP/IP, que será alvo de estudo aprofundado nas aulas teóricas durante o semestre. Sendo assim, qualquer dispositivo conectado à Internet, independentemente da sua natureza (computador, impressora, televisão, *smartphone*, sensor, carro, webcam, etc.), corre software que implementa o TCP/IP. Esta pilha protocolar possibilita que a componente de comunicação das aplicações recorra aos protocolos UDP (*User Datagram Protocol*) e TCP (*Transport Control Protocol*), ambos situados na designada camada de Transporte, mas com características distintas ao nível do serviço oferecido.

Para desenvolver aplicações distribuídas que recorrem aos protocolos UDP e TCP, são geralmente usadas API (*Application Programming Interfaces*) baseadas no conceito de *socket* (“tomada” em inglês). Esta abstração representa um ponto de comunicação através do qual é possível enviar e receber mensagens (cabeçalho + dados) através de uma rede. Linguagens de programação e sistemas distintos possuem API próprias. No caso concreto da programação em linguagem C para

sistemas operativos Windows, recorre-se à API oferecida pela biblioteca de programação Winsock (Windows sockets). São as suas funções, macros e estruturas de dados que irão permitir desenvolver a componente de comunicação na resolução dos exercícios desta ficha.

A Figura 1 apresenta uma visão de alto nível dos principais elementos envolvidos na comunicação entre aplicações via TCP/IP. Na resolução dos exercícios desta ficha, o foco será a utilização da API dos Winsock para a troca de mensagens entre aplicações recorrendo ao protocolo UDP:

- Um *socket* é uma estrutura de dados que representa um *endpoint* de comunicação;
- Através de variáveis deste tipo, é possível enviar e receber dados através de uma rede de dados;
- Como podem existir vários *socket* associados ao mesmo protocolo do nível de transporte no mesmo dispositivo, é necessário poder distingui-los. Para o efeito, cada *socket* está associado a um valor inteiro positivo designado porto.
- Em cada instante, no mesmo sistema, para um determinado protocolo da camada de transporte (UDP ou TCP), um determinado porto não pode estar associado a mais do que um *socket*;
- Na Figura 1, se, por exemplo, a aplicação 1 enviar *bytes* através do seu *socket* UDP indicando como destino o endereço IP 220.20.20.20 (identifica uma interface de rede/dispositivo) e o porto 53, o *datagrama* correspondente acabará por ser colocado no *buffer* de entrada do *socket* da aplicação 3 que está associado ao porto UDP indicado no dispositivo de destino;
- A aplicação 1 também pode comunicar com a aplicação 2 via *sockets*. Neste caso, em vez de indicar como endereço de destino 193.168.10.10, pode usar o endereço de *loopback interface* 127.0.0.1, que identifica a própria máquina;
- Ao receber um *datagrama*, a aplicação de destino pode, além dos dados transportados, extrair deste o endereço IP e o porto do *socket* de origem e, consequentemente, enviar um *datagrama* de volta;
- Uma aplicação pode ter vários *sockets* configurados em simultâneo.

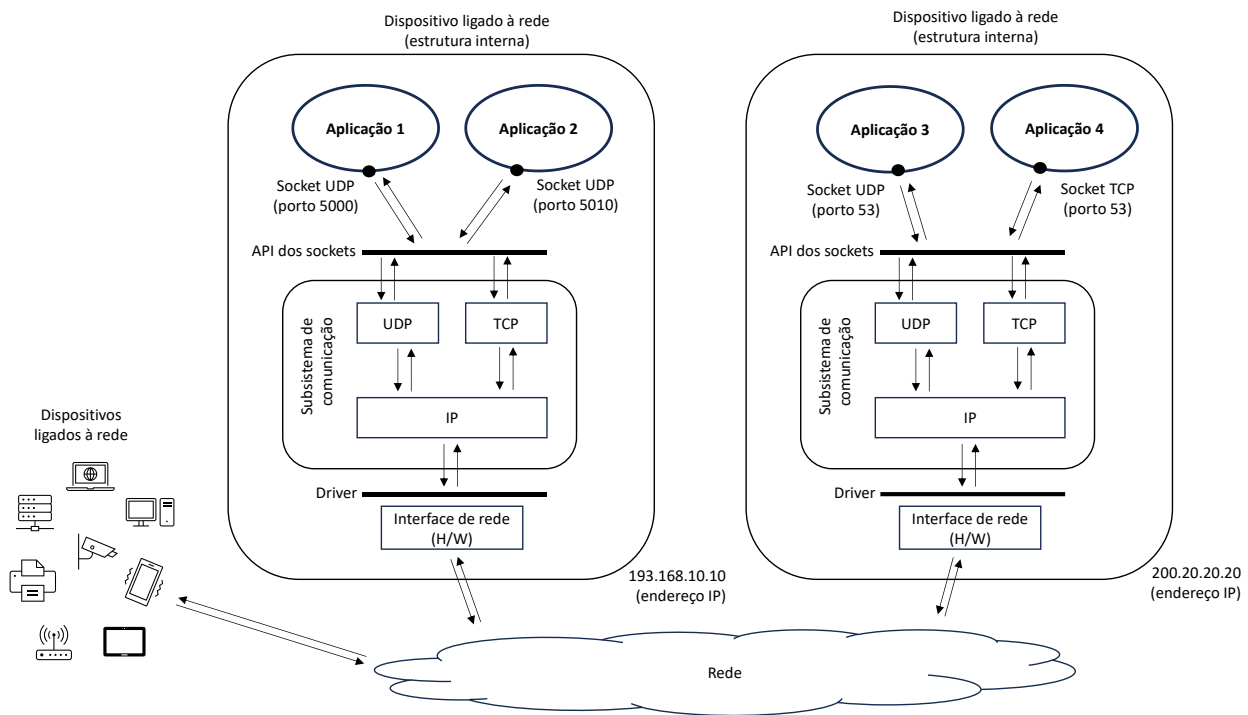


Figura 1 – Comunicação remota via protocolo TCP/IP

A Figura 2 ilustra a sequência elementar de chamadas às funções da biblioteca Winsock em aplicações típicas cliente/servidor que recorrem ao protocolo UDP para interações do tipo pedido/resposta. De um modo geral, em sistemas distribuídos, a aplicação que estabelece o primeiro contacto é o designado cliente e a aplicação que está continuamente a aguardar por contactos é o servidor. A lista dos principais elementos da biblioteca Winsock que serão usados para resolver os exercícios podem ser consultados no final deste documento.

As principais características do protocolo UDP são:

- Um dos dois protocolos da camada de transporte da pilha protocolar TCP/IP;
- Permite a troca de dados (*bytes*) entre aplicações que se situam em dispositivos distintos ligados em rede;
- Envio e receção de pacotes (mensagens) designados *datagramas*;
- É não orientado a ligação;

- Uma aplicação pode, através de um único *socket* UDP, enviar *datagramas* para diversos destinos e receber *datagramas* de diversas origens;
- Não inclui qualquer mecanismo que visa a entrega ordenada, sem duplicação e sem perda dos dados (baixo nível de fiabilidade de comunicação oferecido às aplicações);
- Sobrecarga protocolar reduzida (transmissão de dados mais rápida e menor consumo de recursos, como memória e tempo de CPU, em comparação com o TCP);
- Suporta o envio de dados por difusão.

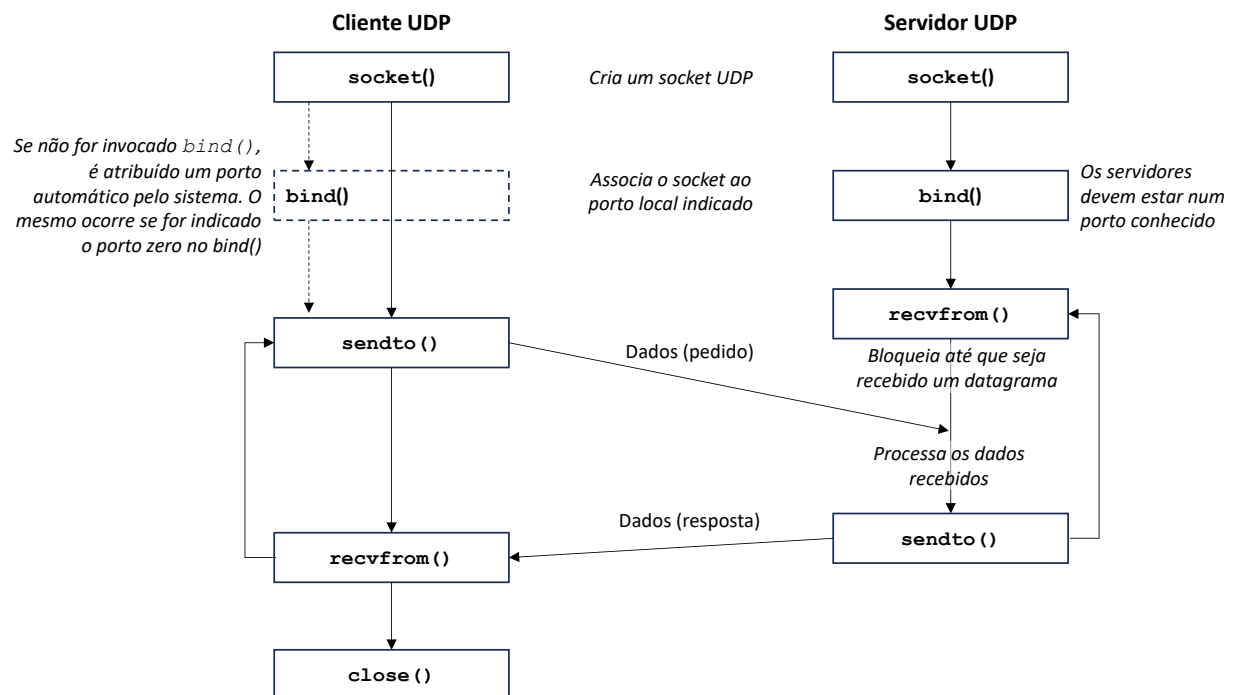


Figura 2 – Comunicação via Winsock UDP

1. Desenvolva uma aplicação cliente/servidor em que os clientes enviam, via protocolo UDP, uma mensagem de texto, passada como argumento através da linha de comando, para um servidor específico com localização dada pelas constantes *SERV_HOST_ADDR* (endereço IP) e *SERV_UDP_PORT* (porto). O servidor deve ir apresentado na saída *standard* as mensagens que vai recebendo.
2. Altere a aplicação anterior de modo a que o servidor reenvie as mensagens recebidas aos respectivos clientes. Estes devem aguardar pelas respostas e apresentá-las na saída *standard*.
3. Altere o cliente do exercício anterior de modo a que este apresente, na saída *standard*, o porto local automaticamente atribuído ao seu *socket* UDP. Esta operação deve ser executada depois do envio da mensagem ao servidor.
4. Altere o servidor desenvolvido no exercício 2 de modo a que este mostre, além dos conteúdos, as localizações de origem das mensagens recebidas.
5. Altere o cliente do exercício 3 para que a localização do servidor seja fornecida através de argumentos passados na linha de comando. Para flexibilizar a ordem pela qual os argumentos são fornecidos, pode recorrer a uma sintaxe baseada em *flags* (por exemplo, -msg “Este exercício é ultra simples” -ip 127.0.0.1 -port 5001).
6. Altere o cliente do exercício anterior de modo a que este verifique se a resposta recebida foi efetivamente enviada pelo servidor.
7. Ponha a correr o servidor desenvolvido em todos os computadores do laboratório, usando o porto de escuta 5001. Invoque, repetidamente, o cliente fornecendo como endereço ip e porto de destino os valores “255.255.255.255” e 5001, respetivamente. Observe e interprete aquilo que acontece.

8. Altere o cliente desenvolvido ao longo dos últimos exercícios de modo a que este aguarde pela resposta do servidor apenas durante um determinado tempo especificado pela constante *TIMEOUT*.
9. Altere o cliente e o servidor que foram desenvolvidos ao longo dos exercícios anteriores de modo a que a resposta do servidor contenha o tamanho da mensagem recebida em formato *ascii*.
10. Altere o cliente e o servidor que foram desenvolvidos ao longo dos exercícios anteriores de modo a que a resposta do servidor contenha o tamanho da mensagem recebida em formato binário. Preveja a possibilidade de ambientes distribuídos heterogéneos.
11. Desenvolva uma aplicação cliente/servidor que obedeça aos seguintes requisitos. O servidor serve de “casamenteiro” entre pares de clientes UDP. Ao receber uma mensagem (o conteúdo é irrelevante) do primeiro cliente (c1), o servidor anota o endereço do mesmo numa estrutura do tipo *struct sockaddr_in*, mas não lhe responde. Quando receber uma mensagem de um segundo cliente (c2), o servidor remete o endereço de c1 a c2, sob a forma de uma variável do tipo *struct sockaddr_in* (em formato binário), dando por concluída a sua tarefa relativamente a este par. O servidor passa, então, a aguardar pelo próximo par de clientes que o contactarem. Depois de enviar uma mensagem ao servidor, qualquer cliente aguarda por uma resposta cujo conteúdo esperado é uma variável do tipo *struct sockaddr_in*. Se a resposta tiver sido enviada pelo servidor, o cliente (i.e., c2) interpreta o seu conteúdo como sendo a localização do seu par (i.e., c1) e reencaminha-a para este. Se o endereço recebido não tiver sido enviado pelo servidor, o cliente (i.e., c1) assume que está a ser contactado pelo seu par (i.e., c2). Em ambas as situações os clientes reportam a localização da sua “cara-metade” na *saída standard* e terminam, dando-se o “nó” por concluído.
12. Desenvolva um servidor UDP que receba, através da linha de comando, um porto de escuta e um nome de utilizador, e reenvie este último à origem de *datagramas* recebidos com conteúdo igual à sequência de caracteres “Hello!”. Desenvolva, igualmente, um cliente

UDP que: (1) receba, através da linha de comando, um porto de destino; (2) configure um *socket* UDP com *timeout* de recessão; (3) envie para este porto e para o endereço IP “255.255.255.255” um *datagrama* com conteúdo igual a “Hello!”; (4) entre num ciclo de receção de *datagramas* até que ocorra uma situação de *timeout*; e (5) para cada *datagrama* recebido, mostre o seu conteúdo bem como a origem (endereço IP e porto) na saída standard.

13. Desenvolva uma aplicação cliente/servidor elementar, baseada no protocolo UDP, que permita obter um ficheiro armazenado no servidor. A aplicação cliente deve ser lançada passando na linha de comando a localização do servidor e o nome do ficheiro pretendido. O servidor deve ser lançado passando, na linha de comando, o porto de escuta e a diretoria local onde se encontram os ficheiros. Deve ser possível transferir ficheiros com qualquer dimensão. Para o efeito, defina um tamanho máximo para os blocos transferidos (por exemplo, MAX_DATA = 4000 bytes), correspondendo o último bloco de um ficheiro a um *datagrama* UDP com conteúdo de tamanho igual a zero *bytes*. O cliente começa por enviar ao servidor um *datagrama* com conteúdo correspondente ao nome do ficheiro pretendido. Qualquer problema que surja durante a transferência de um ficheiro, incluindo situações de *timeout*, leva a que esta seja abortada. Observe os efeitos da ausência de mecanismos de garantia de entrega e de controlo de fluxo do protocolo UDP quando são transferidos ficheiros de grande dimensão.

CRIAÇÃO, ASSOCIAÇÃO A UM PORTO LOCAL E FECHO DE SOCKETS WINDOWS

```
SOCKET socket(int af, int type, int protocol); /* PF_INET, SOCK_DGRAM, IPPROTO_UDP */  
  
int bind(SOCKET s, const struct sockaddr *name, int namelen);  
  
int closesocket(SOCKET s);
```

INDICAÇÃO DE ERRO E CÓDIGOS DE ERRO

```
SOCKET_ERROR  
  
INVALID_SOCKET  
  
int WSAGetLastError(void); /* WSAETIMEDOUT */
```

LOCALIZAÇÃO E CONVERSÃO DE FORMATOS

```
struct sockaddr_in a; /* a.sin_family, a.sin_addr.s_addr, a.sin_port */  
  
u_short htons(u_short hostshort); /* host to network short */  
  
u_long htonl(u_long hostlong); /* host to network long */  
  
u_short ntohs(u_short netshort); /* network to host short */  
  
u_long ntohl(u_long netlong); /* network to host long */  
  
unsigned long inet_addr(const char *cp);  
  
char* inet_ntoa(struct in_addr in); /* network to ascii */
```

ENVIO E RECEPÇÃO DE DATAGRAMAS

```
int sendto(SOCKET s, const char *buf, int len, int flags, struct sockaddr *to, int tolen);  
  
int recvfrom(SOCKET s, char *buf, int len, int flags, struct sockaddr *from, int *fromlen);
```

OBTENÇÃO DE INFORMAÇÃO LOCAL ASSOCIADA AOS SOCKETS

```
int getsockname(SOCKET s, struct sockaddr *name, int *namelen);  
  
int strcmp(const char *s1, const char *s2);  
  
char * strcpy_s(char * strDestination, int sizeStrDestination, const char * strSource);
```

CONFIGURAÇÃO DE OPÇÕES/PARÂMETROS

```
int setsockopt(SOCKET s, int level, int optname, const char *optval, int optlen);  
  
/* level = SOL_SOCKET, optname = SO_RCVTIMEO, optval = (char *)&timeoutMsec (timeoutMsec do tipo  
DWORD) */
```